

Föreläsning – 260623 – ASP.NET CORE WEB.API – Identity, JWT

Identity och **JWT** är två centrala byggstenar inom säkerhet i **ASP.NET Core**, och man brukar gå igenom de tillsammans eftersom de bildar en komplett kedja för autentisering och auktorisering.

1. Identity (Vem är du?)

Detta är grunden och hanterar själva användarregistret.

Det håller reda på **användarnamn**, **lösenord** (hashade), roller och grundläggande profilinformation.

Det är **Identity** som kollar om lösenordet är rätt när användaren försöker logga in.

2. JWT (Ditt pass / Behörighetsbevis)

När Identity har verifierat att användaren är den de utger sig för att vara, skapar systemet en **JWT (JSON Web Token)**.

Token: en fungerar som ett digitalt pass som innehåller information om användaren och dennes behörigheter.

Den signeras digitalt av servern så att den inte kan manipuleras.

Användaren skickar sedan med denna token vid varje anrop för att bevisa sin identitet, utan att servern behöver hålla reda på sessionen i minnet.

3. Policies (Vad får du göra?)

Detta är regelverket. När en användare försöker nå en viss resurs eller ett API, räcker det inte med att bara veta *vem* de är (Identity/JWT). Systemet måste veta vad de **tillåts göra**. Genom Policies kan man sätta upp regler som:

- "För att nå denna endpoint måste användaren vara minst 18 år och tillhöra en specifik avdelning."
- "Denna resurs kräver enbart att användaren har claimet **Permission: Write**"

Det naturliga flödet i praktiken:

När man utvecklar ett system brukar implementeringen se ut så här:

1. **Steg 1 (Registrering/Inloggning):** Du sätter upp **Identity** för att skapa användare.
2. **Steg 2 (Token-generering):** När användaren loggar in konfigurerar man systemet att generera en **JWT** fylld med användarens uppgifter och roller (så kallade *Claims*).
3. **Steg 3 (Skydd & Behörighet):** Man skapar **Policies** i koden som kräver specifika villkor för att ge åtkomst, och applicerar dem sedan på controllers eller endpoints.

Genom att använda dessa tre tekniker tillsammans får man en säker, skalbar och lättunderhållen lösning.

När allt är registrerat i **Program.cs** skyddar man endpoints med attributet **[Authorize]:**

Identity

Identity är inte bara “inloggning”.

Det är **hela infrastrukturen** runt användare, roller, claims och säkerhetsdata. När man kombinerar Identity med JWT i ett Web API får man ett vanligt och modernt sätt att bygga säker API-autentisering.

Identity är Microsofts färdiga system för användare, lösenord, roller och inloggning i en ASP.NET Core-app. Det används för att registrera användare, logga in dem och sedan avgöra vad de får göra via authentication och authorization.

Tänk på Identity som en **personalavdelning för användare**: den skapar konton, sparar lösenord säkert, hanterar roller och hjälper API: t att veta vem som är inloggad. Microsoft beskriver Identity som ett medlemskapssystem som låter oss lägga till login-funktionalitet och lagra användare i till exempel SQL Server.

authentication handlar om att bevisa **vem användaren är**, medan **authorization** handlar om **vad användaren får göra**.

Ett vanligt scenario:

En användare registrerar sig, logga in och får en JWT-token. Sedan kan klienten använda token för att anropa skyddade endpoints, och Swagger kan skicka med token via en Authorize-knapp.

Flödet i ett Web API

I ett Web API med JWT ser flödet ofta ut så här:

Användaren skickar email och lösenord till ett **login-endpoint**, Identity kontrollerar uppgifterna, och om allt stämmer skapas en token som klienten skickar vidare i **Authorization: Bearer ...-headern**.

(När man gör ett API-anrop från till exempel **Swagger**, **Postman** eller en frontend (React, Blazor, MVC) som token skickas i Authorization-headern som Bearer <token>)

JWT används då som ett bevis på att användaren redan har loggat in.

När en endpoint har **[Authorize]** kontrollerar API: t att token finns och är giltig innan den släpper in **requesten**. Microsoft visar också att **[Authorize]** begränsar åtkomst till autentiserade användare, och att role-baserad authorization kan användas för ännu striktare åtkomst.

På serversidan skapar man token i sin AuthController-login-endpoint och returnerar den i svaret. Sedan sparar klienten token och skickar den vidare i headern när den anropar skyddade endpoints.

I Swagger

Om man använder Swagger och trycker på **Authorize**, så är det här man klistrar in token. Swagger skickar sedan automatiskt med den i Authorization-headern för alla test-anrop.

Ett exempel skulle kunna se ut så här:

Projektstruktur

<pre>MyApi ├── Controllers │ ├── AuthController.cs │ └── ProductsController.cs ├── Data │ └── ApplicationDbContext.cs ├── Models │ └── AppUser.cs ├── DTOs │ ├── RegisterRequestDto.cs │ ├── LoginRequestDto.cs │ └── AuthResponseDto.cs ├── Services │ └── TokenService.cs └── Program.cs</pre>	Var i koden det händer • Login händer i din AuthController. • Token skapas av din TokenService. • Skyddad request kontrolleras av AddJwtBearer(...) + [Authorize] i API: t.
--	---

Flödet

1. **Login:** användaren skickar email/lösenord till /api/auth/login.
2. **Token:** API: t verifierar uppgifterna och skapar en JWT.
3. **Skyddad request:** klienten skickar token i headern och API: t validerar den

Klient -> /api/auth/login -> API

API -> JWT-token -> Klient

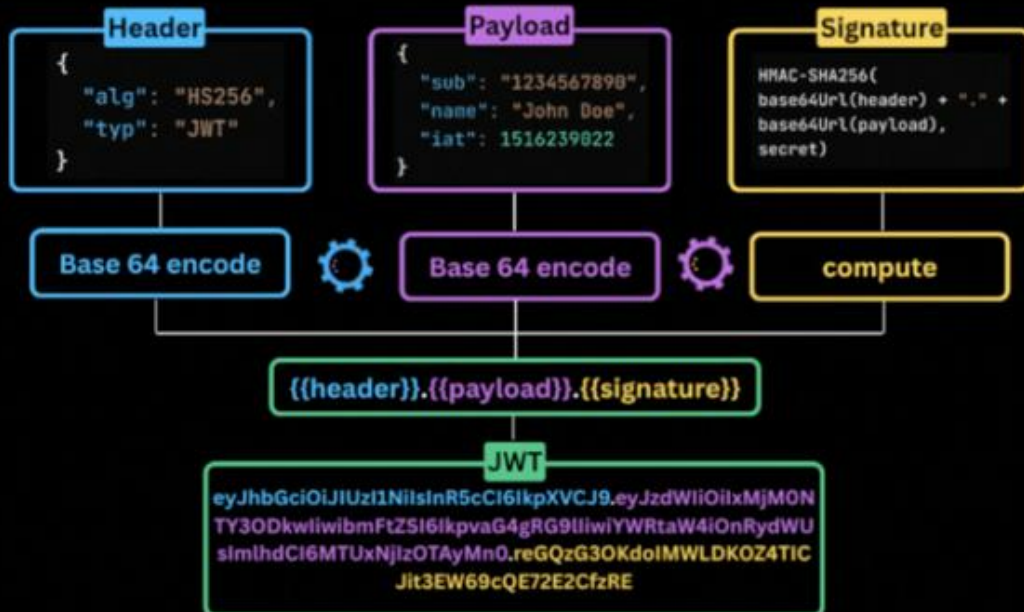
Klient -> Authorization: Bearer <token> -> Skyddad endpoint

API -> 200 OK eller 401 Unauthorized

Vad är JWT?

What is Json Web Token?

Structure of a JWT



How JWT Works?



Understanding JWT Authentication in Modern Web Apps

JWT (JSON Web Token) is a compact, URL-safe way to securely exchange information between a client and a server, most commonly used for authentication in modern web apps.

A JWT consists of three parts separated by dots:

xxxxx.yyyyy.zzzzz Header – Specifies the algorithm (e.g., HMAC SHA256 or RSA) and token type (JWT).

Payload – Contains “claims,” or statements about the user and other data.

Signature – Ensures the token’s integrity by verifying it hasn’t been altered.

How it works:

The user logs in with valid credentials.

The server validates and generates a signed JWT.

The JWT is sent back to the client.

The client stores it (usually in localStorage or cookies).

The client sends the token in the Authorization header for subsequent requests.

Why use JWT?

Stateless: No need for server-side session storage, improving scalability.

Secure: Digitally signed to prevent tampering.

Compact: Lightweight and fast to transmit.